

ATIVIDADE AULA 2

Guia Detalhado para Alunos

*Migrando o Sistema de Cadastro de Vendas de SQLite para Supabase
e criando uma Interface de Consulta em Python/Tkinter*

Sobre este material

Este documento explica, passo a passo, a atividade proposta no slide “ATIVIDADE AULA 2”. O objetivo é que você entenda não apenas O QUE fazer, mas POR QUE cada etapa é necessária, para que consiga resolver o exercício com autonomia e também adaptá-lo a outros projetos no futuro.

1. Visão Geral da Atividade

Na aula anterior, vocês construíram uma interface web (usando a biblioteca Gradio) para um Sistema de Cadastro de Vendas. Naquele momento, todos os dados cadastrados eram salvos em um banco de dados SQLite local — ou seja, um arquivo .db que existia apenas no computador de cada aluno.

Nesta Aula 2, o exercício pede para evoluirmos esse sistema em duas frentes:

- Trocar o banco de dados local (SQLite) por um banco de dados na nuvem, hospedado na plataforma Supabase.
- Criar uma segunda interface, agora rodando como aplicativo desktop (Python + Tkinter), que se conecta a esse MESMO banco de dados na nuvem, mas apenas para visualizar e consultar os registros.

Em outras palavras: ao final do exercício, você terá DUAS interfaces diferentes (uma web, outra desktop) compartilhando UM ÚNICO banco de dados central. Isso é exatamente o que acontece em sistemas profissionais — várias aplicações distintas (site, app mobile, sistema interno) acessando a mesma fonte de dados.

1.1 O enunciado, item a item

1. Retomar a interface web/mobile feita com Gradio (Sistema de Cadastro de Vendas), que antes usava SQLite local.
2. Adaptar essa interface para que ela leia e grave os dados em um banco de dados criado na plataforma Supabase (www.supabase.com), no lugar do SQLite.
3. Criar uma pequena interface IDE, em Python com Tkinter, rodando no computador do aluno, que se conecta ao MESMO banco no Supabase. Essa interface deve ter apenas as funções de visualizar e consultar dados existentes — nada de cadastrar, editar ou excluir.
4. Entregar: (a) o código-fonte completo em Python; (b) o endereço (URL) e a chave (key) do banco de dados Supabase, para que o professor possa testar.

2. Objetivos de Aprendizagem

Ao concluir esta atividade, você terá praticado:

- Integração de uma aplicação Python com um banco de dados na nuvem (PostgreSQL gerenciado pelo Supabase) via API/SDK.
- Separação de responsabilidades entre uma camada de apresentação web (Gradio) e uma camada de apresentação desktop (Tkinter), ambas consumindo a mesma fonte de dados.
- Operações básicas de banco de dados: inserir (INSERT) na interface web e consultar (SELECT) na interface desktop.
- Boas práticas de configuração e segurança de credenciais (URL e chave de API).

3. Parte 1 — Preparando o Banco de Dados no Supabase

O Supabase é uma plataforma que oferece, entre outras coisas, um banco de dados PostgreSQL completo na nuvem, com um plano gratuito suficiente para esta atividade. Siga os passos abaixo.

3.1 Criar conta e projeto

1. Acesse www.supabase.com e crie uma conta gratuita (pode usar login com GitHub ou Google).
2. Clique em “New project”, escolha um nome para o projeto (ex.: cadastro-vendas), defina uma senha forte para o banco de dados (anote-a, ela é diferente da API key) e escolha uma região (ex.: South America - São Paulo).
3. Aguarde alguns instantes enquanto o Supabase provisiona o banco de dados.

3.2 Criar a tabela de vendas

No menu lateral, acesse “Table Editor” e crie uma nova tabela, ou utilize o “SQL Editor” e execute um comando semelhante a este (ajuste os campos conforme os dados que sua interface de cadastro já utilizava):

```
create table vendas (  
  id bigint generated always as identity primary key,  
  produto text not null,  
  quantidade integer not null,  
  valor_unitario numeric(10,2) not null,  
  cliente text,  
  data_venda timestamp with time zone default now()  
);
```

⚠ Atenção — Row Level Security (RLS)

Por padrão, o Supabase ativa o RLS (Row Level Security), que bloqueia qualquer leitura ou escrita até que você crie uma política de acesso. Para esta atividade didática, você tem duas opções:

- 1) Desativar o RLS na tabela (Table Editor > sua tabela > ... > Disable RLS) — mais simples para fins de exercício.
- 2) Criar uma policy permitindo SELECT e INSERT para o público (anon) — mais correto, porém mais avançado.

Se ao testar você receber o erro “new row violates row-level security policy”, é exatamente isso que está acontecendo.

3.3 Obter a URL e a chave (API Key)

1. No menu lateral, acesse “Project Settings” → “API”.
2. Copie o “Project URL” (algo como <https://xxxxxxx.supabase.co>).
3. Copie a “anon public” key (uma chave longa, tipo JWT). É essa URL + chave que você vai usar no código, e também é o que deve ser entregue ao professor.

3.4 Instalar a biblioteca cliente em Python

```
pip install supabase gradio
```

4. Parte 2 — Adaptando a Interface Gradio

A boa notícia é que a estrutura visual da sua interface Gradio (campos de formulário, botão de salvar) não muda. O que muda é apenas a forma como os dados são gravados: em vez de comandos SQL diretos no arquivo .db local, você passa a usar o cliente Python do Supabase.

4.1 Como era antes (SQLite local)

```
import sqlite3

conexao = sqlite3.connect("vendas.db")
cursor = conexao.cursor()

def salvar_venda(produto, quantidade, valor_unitario, cliente):
    cursor.execute(
        "INSERT INTO vendas (produto, quantidade, valor_unitario, cliente) "
        "VALUES (?, ?, ?, ?)",
        (produto, quantidade, valor_unitario, cliente)
    )
    conexao.commit()
    return "Venda registrada com sucesso!"
```

4.2 Como fica com Supabase

```
import gradio as gr
from supabase import create_client

URL_SUPABASE = "https://xxxxxxx.supabase.co"
CHAVE_SUPABASE = "sua_anon_key_aqui"

supabase = create_client(URL_SUPABASE, CHAVE_SUPABASE)
```

```
def salvar_venda(produto, quantidade, valor_unitario, cliente):
    supabase.table("vendas").insert({
        "produto": produto,
        "quantidade": quantidade,
        "valor_unitario": valor_unitario,
        "cliente": cliente
    }).execute()
    return "Venda registrada com sucesso!"

interface = gr.Interface(
    fn=salvar_venda,
    inputs=["text", "number", "number", "text"],
    outputs="text",
    title="Cadastro de Vendas"
)

interface.launch()
```

Note o essencial: tudo o que envolvia sqlite3, conexao e cursor foi substituído pelo objeto supabase, e o comando INSERT INTO em SQL puro foi substituído pelo método `.table("vendas").insert({...}).execute()`. O restante da lógica da interface (campos do formulário, botão, mensagem de retorno) permanece igual.

5. Parte 3 — Criando a Interface Desktop (Tkinter)

Esta segunda interface é um programa independente, que abre uma janela no computador do aluno (não é uma página web) e serve apenas para CONSULTAR os dados que já estão no Supabase — sem cadastrar, editar ou excluir nada. Pense nela como uma “tela de consulta gerencial”.

5.1 O que essa interface deve ter

- Uma tabela (widget Treeview) listando os registros de vendas existentes.
- Um botão “Atualizar lista”, que busca novamente os dados no Supabase.
- Um campo de busca para consultar vendas por algum critério (ex.: nome do produto ou do cliente).
- Nenhum botão de inserir, editar ou excluir — isso é responsabilidade exclusiva da interface Gradio.

5.2 Esqueleto de código

```
import tkinter as tk
from tkinter import ttk
from supabase import create_client

URL_SUPABASE = "https://xxxxxxxxx.supabase.co"
CHAVE_SUPABASE = "sua_anon_key_aqui"
supabase = create_client(URL_SUPABASE, CHAVE_SUPABASE)
```

```

def carregar_dados(filtro=""):
    tabela.delete(*tabela.get_children())
    consulta = supabase.table("vendas").select("*")
    if filtro:
        consulta = consulta.ilike("produto", f"%{filtro}%")
    resposta = consulta.execute()
    for venda in resposta.data:
        tabela.insert("", "end", values=(
            venda["id"], venda["produto"], venda["quantidade"],
            venda["valor_unitario"], venda["cliente"], venda["data_venda"]
        ))

janela = tk.Tk()
janela.title("Consulta de Vendas – Supabase")
janela.geometry("760x420")

topo = tk.Frame(janela)
topo.pack(fill="x", padx=10, pady=10)

campo_busca = tk.Entry(topo, width=30)
campo_busca.pack(side="left", padx=(0, 8))

btn_buscar = tk.Button(topo, text="Consultar",
    command=lambda: carregar_dados(campo_busca.get()))
btn_buscar.pack(side="left", padx=(0, 8))

btn_atualizar = tk.Button(topo, text="Atualizar lista",
    command=lambda: carregar_dados())
btn_atualizar.pack(side="left")

colunas = ("id", "produto", "quantidade", "valor_unitario", "cliente",
    "data_venda")
tabela = ttk.Treeview(janela, columns=colunas, show="headings")
for coluna in colunas:
    tabela.heading(coluna, text=coluna.capitalize())
tabela.pack(fill="both", expand=True, padx=10, pady=(0, 10))

carregar_dados()
janela.mainloop()

```

Esse código é um ESQUELETO funcional: ele já se conecta ao Supabase, carrega os dados em uma tabela visual (Treeview) e permite filtrar por produto. Vocês podem (e devem) adaptar os nomes de colunas, adicionar mais filtros, melhorar o layout, etc. — o importante é manter o princípio de “somente leitura”.

6. Parte 4 — Entregáveis

O exercício deve ser entregue com os seguintes itens:

- Código-fonte completo em Python: tanto o arquivo da interface Gradio (web) quanto o arquivo da interface Tkinter (desktop).
- Endereço (URL) e chave (anon key) do banco de dados Supabase utilizado, para que o professor consiga testar o sistema com os mesmos dados.

Boa prática de segurança (vale a reflexão)

Em projetos reais, NUNCA se compartilha a URL e a chave de um banco de dados dentro do código-fonte que vai para um repositório público (como o GitHub). O correto é guardar essas informações em variáveis de ambiente (arquivo .env) e enviá-las separadamente.

Para esta atividade, como o professor precisa testar o sistema, é aceitável enviar a URL e a chave junto com a entrega — mas pratique já o hábito de não deixar credenciais expostas em projetos que forem publicados.

7. Checklist Final

#	Item	Status
1	Projeto criado no Supabase, com tabela de vendas configurada	☐
2	RLS desativado ou com policy de leitura/escrita configurada	☐
3	Interface Gradio gravando dados no Supabase (não mais no SQLite)	☐
4	Interface Tkinter listando os dados do Supabase	☐
5	Interface Tkinter com função de busca/consulta	☐
6	Interface Tkinter SEM funções de inserir/editar/excluir	☐
7	Código-fonte (.py) de ambas as interfaces organizado	☐
8	URL e anon key do Supabase prontos para envio ao professor	☐

8. Erros Comuns e Dicas de Depuração

“new row violates row-level security policy”

O RLS está ativo e bloqueando a operação. Revise a seção 3.2 (desative o RLS ou crie uma policy).

“invalid API key” ou erro de autenticação

Confira se você copiou a chave “anon public” (e não a “service_role”, que é secreta e não deve ser usada em código cliente) e se não há espaços extras ao colar a URL ou a chave.

A janela do Tkinter abre, mas a tabela aparece vazia

Verifique se a tabela no Supabase realmente tem registros (confira no Table Editor) e se o nome da tabela e das colunas no código Python é exatamente igual ao do banco (atenção a maiúsculas/minúsculas e acentos).

Erro de conexão (timeout, DNS, etc.)

Confirme a URL do projeto (copiada do Project Settings) e se há conexão com a internet. Projetos gratuitos do Supabase podem “pausar” após dias de inatividade — basta acessar o painel do projeto para reativá-lo.

Bons estudos! Em caso de dúvidas durante a implementação, revise cada seção deste guia na ordem em que foi apresentada — a maior parte dos problemas de integração com o Supabase está relacionada às permissões (RLS) ou a credenciais copiadas incorretamente.